

CLIENT-SIDE SECURITY AUDIT REPORT

Target: OpenAI ChatGPT Web Application (SentinelSDK)

Date: July 2026

Auditor: Marcos (servodecristorj-ai)

Status: Responsible Disclosure Forwarded to Engineering

1. INTRODUCTION

During a voluntary and independent analysis of the client-side infrastructure of the ChatGPT web application, I conducted an audit focused on SentinelSDK — a proprietary component developed by OpenAI to manage telemetry, session integrity, and automation (bot) mitigation. The primary objective of this analysis was to evaluate in-memory data isolation and the effectiveness of security policies applied directly in the user's browser.

2. METHODOLOGY AND TECH-STRUCTURAL FINDINGS

A. Global Scope Leakage

Upon inspecting the browser's global scope memory via the `window` object, I identified that the critical state variable `__sentinel_token_pending` was directly exposed and publicly accessible via the developer console (DevTools).

Impact: By not being encapsulated within a closed modular structure or private lexical scope, the variable remains susceptible to reading and monitoring by any third-party script running in the same page execution context, such as browser extensions (malicious or vulnerable) and potential Cross-Site Scripting (XSS) vectors.

[In-Memory Code Evidence]:

```
console.log(window.__sentinel_token_pending);
```

B. Telemetry Traffic Mapping and Component Architecture

Using monitoring hooks in the client's execution environment before process detachment, it was possible to map the continuous flow of analytical data and heartbeat requests directed to the

`/m` and `/ping` endpoints. This evidenced the SDK's persistence behavior and the telemetry transport structure.

Through inspection of the browser's resource tree (Sources), the proprietary structure used by OpenAI to load the system was mapped:

1. The system injects an `iframe` element pointing to the hidden endpoint: `backend-api/sentinel/frame.html`.
2. Under the dynamic versioning directory (e.g., `sentinel/20260423af3c`), the core `sdk.js` file was located, responsible for managing client validation logic.

C. CSP (Content Security Policy) Effectiveness

As a highly positive point, OpenAI's server-side architecture demonstrated extreme robustness. The Content Security Policy (CSP) directives perfectly blocked attempts at code reflection and dynamic string execution (functions like `eval`), mitigating critical direct injection vectors in the browser.

3. MITIGATION PROPOSALS (HOW TO SOLVE)

To correct the global scope leakage identified in the `SentinelSDK` variable, two main architectural approaches are suggested:

Solution 1: Use of IIFE (Immediately Invoked Function Expression) By wrapping the SDK core in a self-executing function, the telemetry state is isolated within the function's scope, preventing it from being attached to the global `window` object:

```
(function() {  
    let __sentinel_token_pending = false; // Restricted to internal scope  
    function handleTelemetry() {  
        // SentinelSDK internal logic  
    }  
})();
```

Solution 2: ES6 Module Scope (EcmaScript Modules) Ensure that the SDK script is loaded strictly as a module (`type="module"`). By default, variables declared at the top level of an ES6 module do not enter the global scope:

```
// sentinel-core.js
let __sentinel_token_pending = false; // Scope restricted to the module f:
export function initSentinel() { ... }
```

4. RESPONSIBLE DISCLOSURE AND OPENAI'S RESPONSE

Following Responsible Disclosure guidelines, the detailed report was sent to the OpenAI security team. Although the finding does not fall under strict financial reward rules (as it involves state exposure in client memory rather than a direct remote execution exploit), the triage response validated the technical relevance of the analysis and confirmed that the feedback was sent to the development team:

"It appears your report is an architectural suggestion rather than a concrete security vulnerability observation, so I'll close this report and route the feedback to the engineering team."

CONCLUSION

Having a technical report analyzed and forwarded directly to OpenAI's engineering team is a significant endorsement. It demonstrates that client-side architecture analyses play a vital role in hardening the barriers of modern complex web applications.